



2019 Fall System Programming

Shell Lab 3

2020. 11. 20

권진세

Embedded System Lab.
Dept. of Computer Science & Engineering
Chungnam National University



실습 소개

- ❖ 과목 홈페이지
 - ◆ 충남대학교 사이버캠퍼스 (<http://e-learn.cnu.ac.kr/>)

- ❖ 실습 서버
 - ◆ Putty 활용
 - ◆ 133.186.221.214

- ❖ 연락처
 - ◆ kjskjsgo@gmail.com



개요

❖ 실습 명

- ◆ Shell Lab - 3

❖ 목표

- ◆ 작업 관리를 지원하는 간단한 Linux Shell 프로그램 구현과 이를 통한 프로세스의 제어와 시그널링(Signalling)의 개념 이해

❖ 과제 진행

- ◆ 수업시간에 배운 유닉스 지식을 활용하여 구현한다.

❖ 구현사항

- ◆ 기본적인 리눅스 셸의 구현
- ◆ 작업 관리(foreground / background)기능 및 셸 명령어 구현



Shell Lab - 3

- ❖ 프로세스의 관리와 시그널의 제어에 대해 이해하고, 작업의 제어를 지원하는 Linux Shell program을 작성하는 것을 목표로 한다.
- ❖ Shell Lab은 **trace08** 부터 **trace12** 까지의 Trace를 모두 수행할 수 있도록 구현되어야 한다.
 - ◆ 테스트 프로그램 'sdriver' 를 통해 테스트할 수 있다.
- ❖ 총 2주에 걸쳐 진행되며, 각 주 별 수행 사항은 아래와 같다.
 - ◆ Shell Lab 1주차 : trace 00, 01, 02
 - ◆ Shell Lab 2주차 : trace 03, 04, 05, 06, 07
 - ◆ Shell Lab 3주차 : **trace 08, 09, 10, 11, 12**



trace00.txt	Properly terminate on EOF.	❖	Shell Lab 01
trace01.txt	Process built-in quit command.		
trace02.txt	Run a foreground job that prints an environment variable		
trace03.txt	Run a synchronizing foreground job without any arguments.	❖	Shell Lab 02
trace04.txt	Run a foreground job with arguments.		
trace05.txt	Run a background job.		
trace06.txt	Run a foreground job and a background job.		
trace07.txt	Use the jobs built-in command.		
trace08.txt	Send fatal SIGINT to foreground job.	❖	Shell Lab 03
trace09.txt	Send SIGTSTP to foreground job.		
trace10.txt	Send fatal SIGTERM (15) to a background job.		
trace11.txt	Child sends SIGINT to itself		
trace12.txt	Child sends SIGTSTP to itself		
trace13.txt	Forward SIGINT to foreground job only.		
trace14.txt	Forward SIGTSTP to foreground job only.		
trace15.txt	Process bg built-in command (one job)		
trace16.txt	Process bg built-in command (two jobs)		
trace17.txt	Process fg built-in command (one job)		
trace18.txt	Process fg built-in command (two jobs)		
trace19.txt	Forward SIGINT to every process in foreground process group		
trace20.txt	Forward SIGTSTP to every process in foreground process group		
trace21.txt	Restart every stopped process in process group		



Shell Lab – Trace 목록 (3주차)

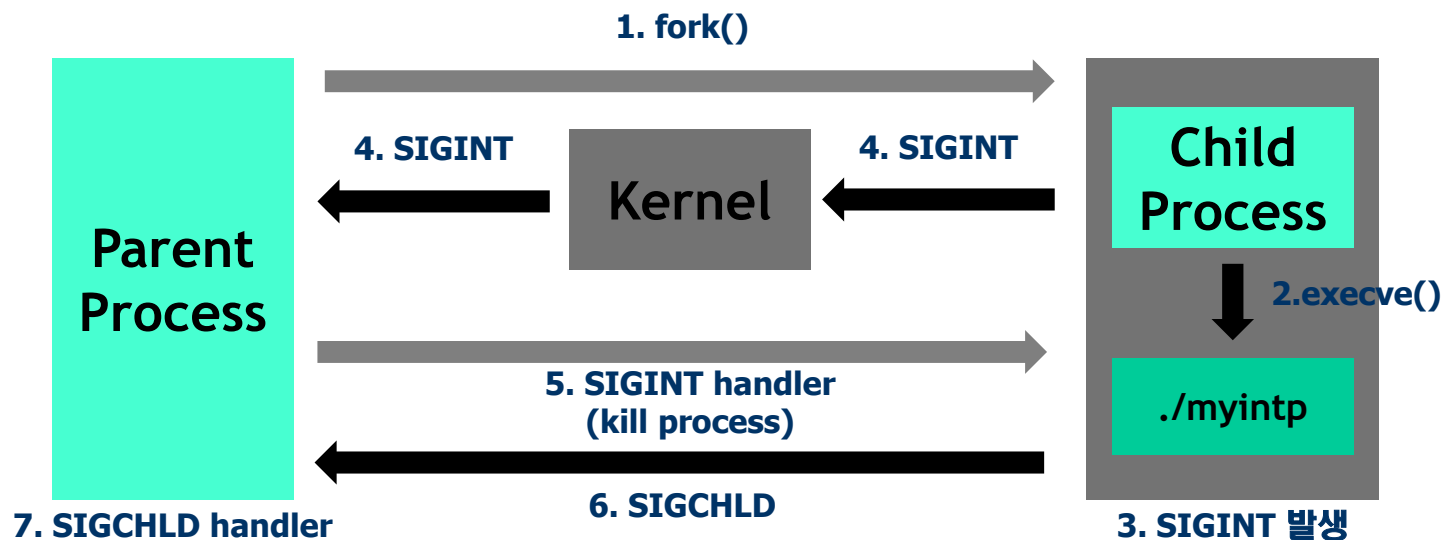
- ❖ sdriver를 이용하여 Trace{08-12}를 테스트 할 수 있다.
 - ◆ 각 trace의 자세한 내용은 해당 파일을 열어서 확인할 수 있다.

Trace	설 명
trace08	자식이 SIGINT를 부모에게 보낼 경우, 자식의 foreground 작업 종료
trace09	자식이 SIGTSTP을 부모에게 보낼 경우, 자식의 foreground 작업 종료
trace10	Background 작업 정상 종료 처리
trace11	자식 프로세스가 스스로에게 SIGINT 전송
trace12	자식 프로세스가 스스로에게 SIGTSTP 전송

Signal의 처리 과정

❖ SIGINT 처리 과정 (trace 08)

- ◆ fork()와 execve() 통해 프로그램 실행
- ◆ SIGINT 발생(ctrl+c)
- ◆ kernel을 통해 자식 프로세스에서 부모 프로세스로 SIGINT 전달 (myintp.c)
- ◆ SIGINT 핸들러를 통해 SIGINT 처리(자식 프로세스 Kill)
- ◆ 자식 프로세스가 종료되면 SIGCHLD 시그널 발생
- ◆ SIGCHLD 핸들러를 통해 자식 프로세스 최종 종료 처리





Signal 목록

- ❖ 각 Signal에 대한 고유 번호가 있다.
- ◆ 아래 그림을 참조

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL
5) SIGTRAP	6) SIGABRT	7) SIGBUS	8) SIGFPE
9) SIGKILL	10) SIGUSR1	11) SIGSEGV	12) SIGUSR2
13) SIGPIPE	14) SIGALRM	15) SIGTERM	16) SIGSTKFLT
17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU
25) SIGXFSZ	26) SIGVTALRM	27) SIGPROF	28) SIGWINCH
29) SIGIO	30) SIGPWR	31) SIGSYS	34) SIGRTMIN
35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3	38) SIGRTMIN+4
39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12
47) SIGRTMIN+13	48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14
51) SIGRTMAX-13	52) SIGRTMAX-12	53) SIGRTMAX-11	54) SIGRTMAX-10
55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7	58) SIGRTMAX-6
59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX		



waitpid(pid_t pid, int *statloc, int options)

❖ Waitpid(①pid_t pid, ②int *statloc, ③int options)

- ◆ ① pid == -1 : 임의의 프로세스 기다림
- ◆ pid > 0 : pid와 일치하는 자식 프로세스를 기다림
- ◆ pid < -1 : 절대값(| pid |)와 그룹 ID가 같은 프로세스를 기다림
- ◆ pid == 0 : waitpid를 호출한 프로세스(부모)와 같은 그룹 ID를 가진 프로세스를 기다림
- ◆ ② 자식프로세스의 종료 status 값을 가져올 포인터를 인자로 넣음.
- ◆ ③ 옵션 - 0 : waitpid(-1,NULL,0) = wait과 같음
- ◆ - WCONTINUED : 중단 되었다가 재개된 자식 프로세스의 상태를 받음
- ◆ - WNOHANG : 기다리는 PID가 종료되지 않아서 즉시 종료 상태를 회수할 수 없는 상황에도 호출자는 멈추지 않고 반환값으로 0을 반환
- ◆ - WUNTRACED : 중단된 자식 프로세스의 상태를 받음



자식프로세스의 종료 상태 확인 매크로

- ❖ WIFEXITED(status) : 자식의 정상 종료 시 True
 - ◆ WEXITSTATUS(status) - exit status를 return

- ❖ WIFSIGNALED(status) : 자식이 Signal을 받아 종료될 경우 True
 - ◆ WTERMSIG(status) - 종료되게 만든 signal 번호를 return
 - ◆ WCOREDUMP(status) - 자식이 core dump를 만들었을 경우 true를 return

- ❖ WIFSTOPPED(status) : 자식이 Stop 될 경우 True
 - ◆ WSTOPSIG(status) - 자식을 멈추게 한 signal 번호를 return

- ❖ WIFCONTINUED(status) : 자식이 SIGCONT 시그널을 받아 다시 실행 될 경우

※ WIF* 와 관련된 W* 매크로간에는 종속관계에 있다.

Example) WIFEXITED일 경우에만 WEXITSTATUS이 정상 작동함.



Shell Lab - trace08

```
#
# trace08.txt - Send fatal SIGINT to foreground job.
#
/bin/echo -e tsh\076 ./myintp
NEXT
./myintp
NEXT

/bin/echo -e tsh\076 quit
NEXT
quit
```

❖ 구현 기능 : SIGINT 발생 시, Foreground 작업 종료

- ❖ SIGINT(키보드의 'ctrl+c' 입력)가 입력 되면 foreground 작업을 kill 한다.
 - ❖ 이전 슬라이드의 시그널 흐름을 이해하는 것이 중요.
- ❖ trace08.txt를 참조하여 동작을 이해한다.
 - ❖ ./myintp 라는 프로그램을 foreground 형태로 실행시킨다.
 - ❖ 해당 프로그램은 SIGINT를 발생시키는 프로그램이다.
 - ❖ SIGINT 발생 결과 프로세스가 종료되고 아래와 같은 형태로 정보를 출력한다.
 - ❖ 'Job [X] (XXXXX) terminated by signal 2'
 - ❖ 'quit'명령을 통해 셸을 종료한다.

❖ HINT

- ❖ waitpid()함수를 통해, 부모 프로세스는 자식 프로세스가 종료되기까지 기다리고, 자식이 종료되면 처리해준다.
- ❖ 자식 프로세스의 종료 상태
 - ❖ WIFEXITED(status) : 정상적으로 자식 프로세스가 종료되었을 경우 0이 아닌 값 리턴 cf. **WEXITSTATUS(status)**
 - ❖ WIFSIGNALED(status) : 자식 프로세스가 어떤 signal에 의해 종료된 경우 TRUE 리턴 cf. **WTERMSIG(status)**
 - ❖ WIFSTOPPED(status) : 자식 프로세스가 정지된 상태라면 TRUE 리턴 cf. **WSTOPSIG(status)**
 - ❖ 위 함수를 이용하여 joblist의 job 정보를 업데이트 해야 한다. 또한 무슨 작업에 의해 종료되었느냐에 따라 적절한 메시지를 출력해야 한다.



Shell Lab - trace08

❖ 구현 방법

- ♦ SIGINT가 발생하면, main() 함수에 등록되어있는 sigint_handler() 함수가 자동으로 호출된다.

```
/* These are the ones you will need to implement */  
Signal(SIGINT, sigint_handler); /* ctrl-c */
```

- ♦ sigint_handler(int sig) 함수 내부에 foreground job을 종료 시키도록 코드를 구현

```
void sigchld_handler(int sig)  
{  
    return;  
}
```



Shell Lab - trace08

■ set 관련 지원함수

- **sigemptyset** - 모든 시그널이 비어 있는 집합 생성
- **sigfillset** - 모든 시그널 번호를 1로 설정
- **sigaddset** - 특정 시그널 번호를 1로 설정
- **sigdelset** - 특정 시그널 번호를 0으로 설정

❖ 구현 방법

- 오른쪽 코드는 관련된 예시 코드입니다.
- 베이스라인 코드로써 수정해야 할 수도 있습니다.
- 빈칸의 코드는 이론 수업의 강의 자료 **syssp-10-signal** 및 교재를 참고하세요.

```

void eval(char *cmdline)
{
    char *argv[MAXARGS];
    pid_t pid;
    int bg;
    sigset_t mask;

    bg = parseline(cmdline, argv);

    [redacted] 시그널 셋 초기화
    [redacted]; //SIGCHLD 시그널을 시그널 셋에 추가
    [redacted]; //SIGINT 시그널을 시그널 셋에 추가
    [redacted] //SIGTSTP 시그널을 시그널 셋에 추가

    [redacted]; //시그널 블록

    if(!builtin_cmd(argv)) {
        if( (pid=fork()) == 0 ) {
            [redacted]; //시그널 언블록
            if( execve(argv[0], argv, environ)< 0 ) {
                printf("%s : Command not found\n", argv);
                exit(0);
            }
        }

        addjob(jobs, pid, (bg ==1?BG: FG), cmdline);
        [redacted]; //시그널 언블록
        if(!bg){
            //부모 프로세서가 자식 프로세서의 종료를 대기 후 처리
            while(1){
                if( pid != fgpid(jobs))
                    break;
                else
                    sleep(1);
            }
        }
        else {
            //백그라운드 작업 수행시 작업 내용 출력
            printf("(%d) (%d) %s", pid2jid(pid), pid, cmdline);
        }
    }

    return;
}

```



Shell Lab - trace08

❖ 구현 방법

- 아래 코드는 관련된 예시 코드입니다. 베이스라인 코드로써 수정해야 할 수도 있습니다.

```
void sigchld_handler(int sig)
{
    int child_status = 0;
    pid_t pid;

    while( [redacted] ) { //자식 프로세스 종료를 대기
        if( [redacted] ) { //자식프로세스를 종료 시킨 시그널이 있는지 체크
            printf("Job [%d] (%d) terminated by signal %d\n", pid2jid(pid), pid, WTERMSIG(child_status));
            [redacted] //자식 프로세스를 목록에서 제거
        }
        else {
            deletejob(jobs, pid);
        }
    }
    return;
}

void sigint_handler(int sig)
{
    [redacted] //포어그라운드 프로세스의 pid 저장
    if(pid != 0){
        [redacted] //프로세스에 시그널 전달
    }
    return;
}
```



Shell Lab - trace08

❖ 결과 확인

- trace08를 sdriver를 수행하여 결과를 확인한다.

- ❖ 명령어 : `./sdriver -t 08 -s ./tsh -V`
- ❖ 이때 tsh 수행 결과가 tshref와 다르다면,

```
Running trace08.txt...  
Oops: test and reference outputs for trace08.txt differed.
```

- ❖ 같다면,

```
Running trace08.txt...  
Success: The test and reference outputs for trace08.txt matched!
```

❖ HINT

- 다음 System Call을 이용하여 다른 프로세스로 시그널을 전달한다.

```
int kill(pid_t pid, int sig);  
Return 성공시 0, 에러시 -1
```

- ❖ pid가 0보다 크면, kill 함수는 지정한 pid를 가지는 프로세스에만 시그널을 전달
- ❖ pid가 0보다 작으면, pid의 절대값 프로세스 그룹에 속하는 모든 프로세스에 시그널을 전달
- ❖ 참조 : <http://i1004me2.blog.me/140190173943>



Shell Lab - trace09

- ❖ **구현 기능** : SIGTSTP 발생 시 Foreground 작업 정지
 - ◆ SIGTSTP(키보드의 'ctrl+z' 입력)이 되면 foreground 작업을 stop 한다.
 - ❖ 이전 슬라이드의 시그널 흐름을 이해하는 것이 중요.
 - ◆ trace09.txt를 참조하여 동작을 이해한다.
 - ❖ ./mytstp 라는 프로그램을 foreground 형태로 실행시킨다.
 - ◆ 해당 프로그램은 SIGTSTP를 발생시키는 프로그램이다.
 - ❖ SIGTSTP 발생 결과 프로세스가 정지되고 아래와 같은 형태로 정보를 출력한다.
 - ◆ 'Job [X] (XXXXX) stopped by signal 20'
 - ❖ 'jobs' 명령을 통해 아래와 같이 정보를 출력한다.
 - ◆ '(X) (XXXXX) Stopped ./mytstp'

```
#
# trace09.txt - Send SIGTSTP to foreground job.
#
/bin/echo -e tsh\076 ./mytstp
NEXT
./mytstp
NEXT

/bin/echo -e tsh\076 jobs
NEXT
jobs
NEXT

quit
```



Shell Lab - trace09

❖ 구현 방법

- SIGTSTP 시그널을 받아서 joblist 에서 해당 job의 상태를 stop으로 변경 해주어야 한다.
- trace08의 구현 방법과 마찬가지로,
- **sigchld_handler(), sigtstp_handler() 함수의 내용을 구현해야 한다.**
- **sigchld_handler() 함수 내부에서 joblist를 체크해 해당 job의 상태를 stop으로 변경하고 정지시켜야 한다.**

```
/* getjobpid - Find a job (by PID) on the job list */
struct job_t *getjobpid(struct job_t *jobs, pid_t pid) {
    int i;

    if (pid < 1)
        return NULL;
    for (i = 0; i < MAXJOBS; i++)
        if (jobs[i].pid == pid)
            return &jobs[i];
    return NULL;
}
```

❖ 결과 확인

- trace09를 sdriver로 결과를 확인
 - ❖ 명령어 : ./sdriver -t 09 -s ./tsh -V

❖ HINT

- trace09도 trace08번과 동일하게 kill()함수를 사용한다.

```
/* Job states */
#define UNDEF 0 /* undefined */
#define FG 1 /* running in foreground */
#define BG 2 /* running in background */
#define ST 3 /* stopped */
```

```
struct job_t {
    pid_t pid; /* job PID */
    int jid; /* job ID [1, 2, ...] */
    int state; /* UNDEF, BG, FG, or ST */
    char cmdline[MAXLINE]; /* command line */
};
```



Shell Lab - trace10

- ❖ **구현 기능** : Background 작업 정상 종료 처리
 - ◆ Background 작업이 정상 종료되면 SIGCHLD가 발생하는데, 이 시그널을 받고 Background 작업을 종료 처리해준다.
 - ❖ 이때, 핸들러에서는 종료된 자식 프로세스가 사용했던 자원을 반환 시켜주어야 한다.
 - ◆ trace10.txt를 참조하여 동작을 이해한다.
 - ❖ ./myspin1 이라는 프로그램을 Background 형태로 5초 동안 실행시킨다.
 - ❖ Background 작업이 종료되기 전, /bin/kill 을 통해 SIGTERM을 발생시킨다.
 - ◆ SIGTERM은 정상 종료 시그널
 - ❖ 작업이 종료되면, 아래와 같이 출력 후 joblist에서 제거된다.
 - ◆ 'Job [X] (XXXXX) terminated by signal 15'

```
#
# trace10.txt - Send fatal SIGTERM (15) to a background job.
#
/bin/echo -e tsh\076 ./myspin1 5 \046
NEXT
./myspin1 5 &
NEXT

WAIT

/bin/echo -e tsh\076 /bin/kill myspin1
NEXT
/bin/kill myspin1
NEXT

/bin/echo -e tsh\076 quit
NEXT
quit
```



Shell Lab - trace10

❖ 구현 방법

- ◆ sigchld_handler() 함수의 내부를 구현
- ◆ 자식 프로세스가 정상 종료되고, joblist 에 그것을 반영해야 한다.

❖ 결과 확인

- ◆ trace10을 sdriver를 수행하여 결과를 확인한다.
 - ❖ 명령어 : ./sdriver -t 10 -s ./tsh -V

❖ HINT

- ◆ waitpid()함수를 통해, 부모 프로세스는 자식 프로세스가 종료되기까지 기다리고, 자식이 종료되면 처리해준다.
- ◆ 자식 프로세스의 종료 상태
 - ❖ WIFEXITED(status) : 정상적으로 자식 프로세스가 종료되었을 경우 0이 아닌 값 리턴
 - ❖ WIFSIGNALED(status) : 자식 프로세스가 어떤 signal에 의해 종료된 경우 TRUE 리턴
 - ❖ WIFSTOPPED(status) : 자식 프로세스가 정지된 상태라면 TRUE 리턴
 - ❖ 위 함수를 이용하여 joblist의 job 정보를 업데이트 해야 한다. 또한 무슨 작업에 의해 종료되었느냐에 따라 적절한 메시지를 출력해야 한다.



Shell Lab - trace11~12

trace11

- ❖ **구현 기능** : 자식 프로세스 스스로에게 SIGINT 전송되고 처리
 - ◆ 자신의 pid(자식 프로세스의 pid)로 SIGINT를 전달하였을 때, 정상적으로 SIGINT에 대한 처리가 되도록 구현
 - ◆ trace08를 제대로 구현했다면 자동으로 통과된다.
 - ❖ 자동으로 통과되는 이유를 생각해보고 보고서에 작성.

trace12

- ❖ **구현 기능** : 자식 프로세스 스스로에게 SIGTSTP 전송되고 처리
 - ◆ 자신의 pid(자식 프로세스의 pid)로 SIGTSTP를 전달하였을 때, 정상적으로 SIGTSTP에 대한 처리가 되도록 구현
 - ◆ trace09를 제대로 구현했다면 자동으로 통과된다.
 - ❖ 자동으로 통과되는 이유를 생각해보고 보고서에 작성.



주의 사항

- ❖ Shell Lab을 수행하다 보면, 부모 프로세스가 자식 프로세스보다 먼저 종료되어 자식 프로세스가 **좀비**가 되는 경우가 있다.

- ◆ 셸 종료 후, 'ps' 명령어를 입력

```
[b000000000@eslab shlab-handout]$ ps
  PID TTY          TIME CMD
 7714 pts/1        00:00:00 bash
 7805 pts/1        00:00:00 ps
31338 ?              00:00:00 tsh <defunct>
```

- ◆ 위와 같이 tsh가 좀비가 되어 남아있는 것을 볼 수 있다. 따라서 해당 좀비 프로세스를 제거해야 한다.
 - ❖ **kill -9 <PID>**
 - ❖ ex) kill -9 31338
- ◆ 좀비 프로세스를 많이 만들면 감점! (셸 테스트 후 실시간 체크 바람)



과제

1. Shell Lab

- I. trace00 ~ trace12에 대한 코드 작성

2. Shell Lab 보고서

- I. 각 trace 별, tshref를 수행한 결과와 본인이 구현한 tsh와의 동작 일치를 증명
 - 1) sdriver 수행 결과와 tsh에서의 정상작동 모습(-v 옵션 사용)
- II. 각 trace 별, 플로우 차트
 - 1) 간단한 수행과정을 플로우차트로 나타내면 됨.
- III. 각 trace 별, 해결 방법에 대한 설명

3. 사이버캠퍼스

- I. 사이버캠퍼스를 통해서 제출하기 바람.
- II. 파일 제목: [sys02]ShellLab_학번_이름.zip

4. 제출일자

- I. 사이버 캠퍼스: 2020년 11월 27일 금요일 10시 00분 00초까지